

ProDrive IQ — Project Report

Milestone 2: Functional Web Application

BIS351 — Web Business Applications

Team Member	Student ID
Mansour Alkhater	2240004171
Sultan Alqasim	2240007388
Faisal Alsaleh	2240004662
Abdullah Alsinan	2240005976

Course: BIS351 — Web Business Applications

Milestone: 2 — Functional Web Application

Instructor: Dr. Nabeel Almushasha

Section: 3102

Contents

Contents	2
1. Business Problem	3
2. Solution Overview	3
3. Target Users.....	3
4. System Pages and Features	3
4.1 Home Page (index.html).....	3
4.2 Login / Register Page (login.html)	4
4.3 Service Providers Page (providers.html)	4
4.4 Booking Page (booking.html)	4
4.5 User Dashboard (dashboard.html).....	4
4.6 About Page (about.html)	4
5. Technical Implementation.....	5
CSS Architecture	5
6. JavaScript Functionality	6
7. Data Handling (localStorage).....	7
8. File Structure	7
9. Development Process.....	8
10. GitHub Links.....	9
11. Conclusion	9

1. Business Problem

Car owners in Saudi Arabia face several problems with car service management. Finding trustworthy and reliable workshops is extremely difficult — most people rely on word-of-mouth or haphazard internet searches. Lack of price transparency is another prevalent issue; customers typically do not know the cost of services such as an oil change, brake repair, or routine maintenance until they physically visit a workshop. The final issue is the inefficiency of the booking process, which requires calling or visiting the workshop in person, often resulting in waiting times and inconvenience.

These problems highlight the need for a unified digital platform that makes it easy for car owners to find, evaluate, and book car services.

2. Solution Overview

ProDrive IQ is a web-based car service booking platform built for car owners across Saudi Arabia. The platform allows users to:

- Browse verified car workshops across four major cities (Riyadh, Jeddah, Dammam, Al Khobar)
- Filter workshops by city and service type (Mechanical, Maintenance, Body & Paint, Electrical)
- View workshop ratings and starting prices in SAR
- Book service appointments online with a full cost breakdown including 15% VAT
- Manage bookings and account information through a personal dashboard

3. Target Users

The primary target audience for ProDrive IQ is:

- Car owners residing in Saudi Arabia
- Individuals seeking trusted, affordable, and transparent car maintenance services
- Users who want a quick and hassle-free way to book service appointments online

The platform is designed for general consumers rather than commercial fleets, with a simple and intuitive interface suitable for any car owner regardless of technical background.

4. System Pages and Features

ProDrive IQ consists of six pages, each serving a specific function in the user journey.

4.1 Home Page ([index.html](#))

The landing page introduces the platform and displays six featured workshop cards. Each card shows the workshop name, location, rating, starting price, and a "Book Now" button. A search bar at the top allows users to search by keyword and filter by city, redirecting to the providers page with the filter pre-applied.

4.2 Login / Register Page (login.html)

A single page that handles both login and registration. The form toggles between two modes using JavaScript. Registration requires a full name, email, password (minimum 8 characters, one uppercase, one number), and password confirmation. Inline validation provides real-time feedback. On successful registration, credentials are saved to localStorage and the user is redirected to the dashboard. If a user attempts to book without being logged in, they are redirected here automatically, then sent back to the booking page after authentication.

4.3 Service Providers Page (providers.html)

Displays all 10 verified workshops as cards rendered dynamically by JavaScript from a central data array. Users can filter by city using a dropdown and by service type using checkboxes. Filters apply instantly without a page reload. Each card includes a "Book" button that initiates the booking flow.

4.4 Booking Page (booking.html)

A multi-step booking form where users:

- Select a service (Oil Change, Tire Rotation, Brake Inspection, etc.)
- Pick a date from dynamically generated chips (next 7 days)
- Select a time slot
- Enter their vehicle make, model, and year

A live summary panel on the right updates in real time showing the selected service, schedule, vehicle, subtotal, 15% VAT, and total in SAR. Confirming the booking saves it to localStorage and redirects to the dashboard with a confirmation toast notification.

4.5 User Dashboard (dashboard.html)

A protected page (redirects to login if no session exists) with a sidebar for navigation between three sections:

- Overview — stat cards (total spent, booking count, account status) and upcoming bookings list
- History — past services section
- Profile — displays account name and email

4.6 About Page (about.html)

A static informational page covering the company story, platform statistics (10+ workshops, 4 cities, 500+ bookings, 4.8 average rating), a problem/solution comparison, a three-step "How It Works" guide, core company values, and contact information.

5. Technical Implementation

The application is built entirely with front-end technologies — no server or database is required.

Technology	Usage
HTML5	Semantic page structure using <nav>, <main>, <section>, <aside>, <footer>
CSS3	All styles written from scratch in css/style.css — no frameworks or CDN dependencies
JavaScript (ES5)	All interactivity and logic in a single shared file js/script.js
localStorage	Client-side data persistence for users, sessions, and bookings

CSS Architecture

All styles are written from scratch in style.css with no external frameworks. The file is organized into two layers:

Component classes — reusable UI elements used across all pages:

- .btn-primary — primary action button
- .card — white card with ambient shadow
- .page-input — styled form input with focus ring and validation states
- .nav-link — navigation anchor styling
- .badge — small rating/status pill
- .glass-nav — frosted glass fixed navigation bar

Layout classes — responsive structural patterns:

- .grid-3, .grid-4, .grid-stats — responsive CSS Grid layouts (1 → 2 → 3/4 columns)
- .sidebar-layout / .sidebar — flex row on desktop, stacked on mobile
- .booking-layout / .summary-col — booking page two-column layout
- .sticky-sidebar — position sticky for sidebar panels on desktop
- .flex-between — flex row with space-between alignment
- .page-container — centered max-width content wrapper

This approach keeps the HTML clean and readable while making the CSS fully self-contained and explainable without any framework knowledge.

6. JavaScript Functionality

All JavaScript for the entire site is contained in a single file: `js/script.js`. The file is organized into eight clearly labelled sections:

Section	Purpose
A — Validation Utilities	<code>validateEmail()</code> , <code>validatePassword()</code> , <code>showError()</code> , <code>clearError()</code>
B — Global Data	<code>workshops[]</code> array, <code>goToBooking()</code> , <code>selectChip()</code> helper
C — Navigation	<code>updateNav()</code> — shows Login or Dashboard button based on session
D — Home Page	Search bar and keyword/city redirect to providers page
E — Providers Page	<code>getFilteredWorkshops()</code> , <code>renderWorkshops()</code> , filter event listeners
F — Booking Page	<code>buildDateChips()</code> , <code>updateSummary()</code> , confirm booking logic
G — Dashboard	<code>loadUser()</code> , <code>renderBookings()</code> , sidebar section switching, logout
H — Login/Register	<code>applyMode()</code> , form toggle, registration, login with credential check

Page detection pattern: Each section only runs when its target page is loaded, using:

```
if (document.getElementById('searchBtn')) { /* home page logic */ }
if (document.getElementById('confirmBtn')) { /* booking page logic */ }
```

Key functions:

`goToBooking(id)`

Called from any "Book" button across the site. Checks if the user is logged in; if not, saves the intended workshop to `localStorage` and redirects to login, then restores the booking flow after authentication.

`selectChip(selector, clickedEl, callback)`

A reusable helper that deselects all elements matching a selector and selects the clicked one. Used for service cards, date chips, and time chips on the booking page.

`renderBookings(list, containerId, isPast)`

Renders a list of bookings into a container. The `isPast` flag switches between a detailed card view (upcoming) and a compact invoice row (history).

7. Data Handling (localStorage)

The application uses the browser's localStorage as its data layer. No backend or database is required.

Key	Data Stored
user	Logged-in user object: { name, email }
registeredUsers	Array of all registered accounts: [{ name, email, password }]
selectedProvider	Workshop object chosen for booking
pendingProvider	Workshop saved temporarily when user is redirected to login
bookings	Array of all confirmed booking objects

Authentication flow:

- Registration saves credentials to registeredUsers[]
- Login checks the submitted email and password against registeredUsers[]
- On success, a user session object is written and the user is redirected
- Logout removes the user key, ending the session

8. File Structure

```
FINAL/
├── index.html           Home page
├── login.html           Login and registration
├── providers.html       Workshop listing with filters
├── booking.html         Appointment booking flow
├── dashboard.html       User dashboard
├── about.html           Company information
├── css/
│   └── style.css        All custom styles – components, layout, responsive
design
├── js/
│   └── script.js        All JavaScript for the site
└── images/
    ├── hero-bg.jpg
    ├── workshop-1.jpg
    └── workshop-2.jpg ... workshop-10.jpg
```

9. Development Process

Step 1 — Problem Identification: Identifying common problems faced by car owners in Saudi Arabia: difficulty finding workshops, lack of price transparency, and inefficient booking methods.

Step 2 — Idea Generation: Deciding on a web application with core features: workshop discovery, price comparison, and online appointment booking.

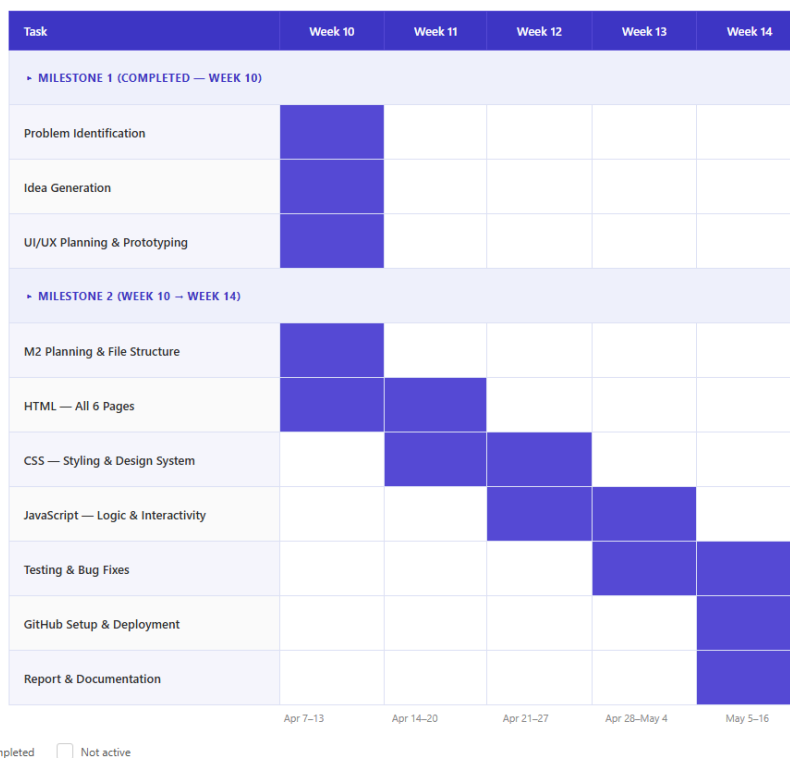
Step 3 — UI/UX Planning (Milestone 1): Defining the six main pages and designing screen prototypes using a UI/UX design tool. Prototypes were reviewed and refined for usability and consistency.

Step 4 — Implementation (Milestone 2): Building the fully functional application using HTML, CSS, and JavaScript. All six pages were implemented with working navigation, forms, dynamic content, and localStorage data handling.

Step 5 — Testing and Refinement: Testing the full user journey end-to-end: registering an account, browsing workshops, booking a service, viewing the booking in the dashboard, and logging out. Edge cases such as booking without logging in and form validation errors were verified.

Below is the updated Gantt chart for Milestone 2.

ProDrive IQ – Project Schedule (Updated)



10. GitHub Links

Repository	Link	Purpose
Live Website	https://mbm0008.github.io/pro-drive-iq-live/	Deployed via GitHub Pages
Project Files	https://github.com/mbm0008/prodrive-iq/tree/milestone-2	Full source code (milestone-2 branch)

11. Conclusion

ProDrive IQ successfully addresses the identified car service problems in Saudi Arabia by providing a centralized platform for discovering and booking verified workshops. The Milestone 2 implementation delivers a fully functional prototype with real user authentication, dynamic content rendering, a complete booking flow with VAT calculation, and a personal dashboard — all built with clean, readable, and well-organized front-end code. The project demonstrates effective use of HTML, CSS, JavaScript, and localStorage to create a real-world business web application.